Day 4

1.Given an array containing only 0,1 and 2 sorted without using any builtin sort function.

```
#include <iostream>
using namespace std;

void sort012(int arr[], int n) {
   int low = 0, mid = 0, high = n - 1;

   while (mid <= high) {
      if (arr[mid] == 0) {
         swap(arr[low], arr[mid]);
         low++;
         mid++;
      }
      else if (arr[mid] == 1) {
         mid++;
      }
      else {
         swap(arr[mid], arr[high]);
         high--;
      }
   }
}

int main() {
   int arr[] = {0, 2, 1, 2, 0, 1, 0, 2};
   int n = sizeof(arr) / sizeof(arr[0]);
```

```
    sort012(arr, n);


    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }


    return 0;
}
```

Output:

```
D:\cpp\day 4\1.exe

Sorted array: 0 0 0 1 1 2 2 2
--------------------------------
Process exited after 2.253 seconds with return value 0
Press any key to continue . . .
```

2.Given an array and integer k, rotate the array to the right by k steps efficiently.

```
#include <iostream>
using namespace std;


void reverseArray(int a[], int start, int end) {
    while (start < end) {
        int temp = a[start];
        a[start] = a[end];
        a[end] = temp;
        start++;
        end--;
    }
```

```
}

int main() {
    int n, k;
    cout << "Enter size: ";
    cin >> n;

    int a[100];
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    cout << "Enter k: ";
    cin >> k;

    k = k % n;

    reverseArray(a, 0, n - 1);
    reverseArray(a, 0, k - 1);
    reverseArray(a, k, n - 1);

    cout << "Array after right rotation: ";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }

    return 0;
```

}

Output:

```
Enter size: 5
Enter array elements: 1
2

3
4
5
Enter k: 2
Array after right rotation: 4 5 1 2 3
--------------------------------
Process exited after 12.12 seconds with return value 0
Press any key to continue . . .
```

3.Given a string of bracket like "({[]})",ensure the list as a stack to determine if the string is balanced.

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;


int main() {

   string s;

   cout << "Enter bracket string: ";

   cin >> s;


   stack<char> st;

   bool balanced = true;


   for (int i = 0; i < s.length(); i++) {

      char ch = s[i];
```

```
    if (ch == '(' || ch == '{' || ch == '[') {
        st.push(ch);
    } else {
        if (st.empty()) {
            balanced = false;
            break;
        }

        char top = st.top();
        st.pop();

        if ((ch == ')' && top != '(') ||
            (ch == '}' && top != '{') ||
            (ch == ']' && top != '[')) {
            balanced = false;
            break;
        }
    }
}

if (!st.empty()) {
    balanced = false;
}

if (balanced)
    cout << "Balanced";
else
    cout << "Not Balanced";
```

```
    return 0;
}
```

Output:

```
Enter bracket string: {[]}
Balanced
--------------------------------
Process exited after 8.558 seconds with return value 0
Press any key to continue . . .
```

4.Given tokens of an RPN expression (eg:["2","1","+","3","*"]),use the list as a stack to compute the results.

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;


int main() {

    int n;

    cout << "Enter number of tokens: ";

    cin >> n;


    string tokens[100];

    cout << "Enter tokens: ";

    for (int i = 0; i < n; i++) {

        cin >> tokens[i];

    }
```

```
    stack<int> st;

    for (int i = 0; i < n; i++) {
        string token = tokens[i];

        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();

            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
            else if (token == "/") st.push(a / b);
        } else {
            st.push(stoi(token));
        }
    }

    cout << "Result = " << st.top();

    return 0;
}
```

Output:

```
Enter number of tokens: 5
Enter tokens: 2
1

+
3
*
Result = 9
--------------------------------
Process exited after 20.33 seconds with return value 0
Press any key to continue . . .
```

5.For a circular array A(the element after the last is the 1st),compute the next greater element foe each index using stack.

```
#include <iostream>

#include <stack>

using namespace std;


int main() {

  int n;

  cout << "Enter size: ";

  cin >> n;


  int a[100], result[100];

  cout << "Enter elements: ";

  for (int i = 0; i < n; i++) {

    cin >> a[i];

    result[i] = -1;

  }


  stack<int> st;
```

```
    for (int i = 2 * n - 1; i >= 0; i--) {
        while (!st.empty() && st.top() <= a[i % n]) {
            st.pop();
        }

        if (i < n) {
            if (!st.empty()) {
                result[i] = st.top();
            }
        }

        st.push(a[i % n]);
    }

    cout << "Next greater elements: ";
    for (int i = 0; i < n; i++) {
        cout << result[i] << " ";
    }

    return 0;
}
```

Output:

```
Enter size: 5
Enter elements: 1
2
3
4
5
Next greater elements: 2 3 4 5 -1
--------------------------------
Process exited after 7.185 seconds with return value 0
Press any key to continue . . .
```

6.implement a text editor undo/redo using stack.

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;


int main() {

  stack<string> undoStack;

  stack<string> redoStack;


  string text = "";

  int choice;

  string newText;


  do {

    cout << "\n1. Type Text\n2. Undo\n3. Redo\n4. Show Text\n5. Exit\n";

    cout << "Enter choice: ";

    cin >> choice;

    cin.ignore();


    switch (choice) {
```

```
case 1:
    undoStack.push(text);
    cout << "Enter new text: ";
    getline(cin, newText);
    text += newText;
    while (!redoStack.empty()) redoStack.pop();
    break;

case 2:
    if (!undoStack.empty()) {
        redoStack.push(text);
        text = undoStack.top();
        undoStack.pop();
    } else {
        cout << "Nothing to undo\n";
    }
    break;

case 3:
    if (!redoStack.empty()) {
        undoStack.push(text);
        text = redoStack.top();
        redoStack.pop();
    } else {
        cout << "Nothing to redo\n";
    }
    break;
```

```
        case 4:
            cout << "Current Text: " << text << endl;
            break;

        case 5:
            cout << "Exiting...\n";
            break;

        default:
            cout << "Invalid choice\n";
    }

  } while (choice != 5);

  return 0;
}
```

OUTPUT:

```
1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 1
Enter new text: JOYCE

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 4
Current Text: JOYCE

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 2

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 4
```

```
Current Text:

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 3

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 4
Current Text: JOYCE

1. Type Text
2. Undo
3. Redo
4. Show Text
5. Exit
Enter choice: 5
Exiting...

--------------------------------
Process exited after 35.27 seconds with return value 0
Press any key to continue . . .
```

7) Given an array and a window size scale, using a dequeue to find the maximum element in each sliding window.

```
#include <iostream>

#include <deque>

using namespace std;



int main() {

   int n, k;

   cout << "Enter size of array: ";

   cin >> n;



   int a[100];
```

```
cout << "Enter elements: ";
for (int i = 0; i < n; i++) {
    cin >> a[i];
}

cout << "Enter window size: ";
cin >> k;

deque<int> dq;

cout << "Sliding window maximums: ";
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() <= i - k) {
        dq.pop_front();
    }

    while (!dq.empty() && a[dq.back()] < a[i]) {
        dq.pop_back();
    }

    dq.push_back(i);

    if (i >= k - 1) {
        cout << a[dq.front()] << " ";
    }
}

return 0;
```

}

Output:

```
Enter size of array: 5
Enter elements: 10
20
30
40
50
Enter window size: 3
Sliding window maximums: 30 40 50
--------------------------------
Process exited after 12.94 seconds with return value 0
Press any key to continue . . .
```

8) Simulate a round robin CPU scheduling using a circular queue.

```
#include <iostream>

using namespace std;


struct Process {

   int id;

   int burstTime;

};


int main() {

   int n, tq;

   cout << "Enter number of processes: ";

   cin >> n;


   Process p[100];

   for (int i = 0; i < n; i++) {

      p[i].id = i + 1;

      cout << "Enter burst time for Process " << p[i].id << ": ";

      cin >> p[i].burstTime;
```

```
}

cout << "Enter time quantum: ";
cin >> tq;

int rem[100];
for (int i = 0; i < n; i++) {
   rem[i] = p[i].burstTime;
}

int time = 0;
bool done;

cout << "\nExecution Order:\n";

do {
   done = true;

   for (int i = 0; i < n; i++) {
      if (rem[i] > 0) {
         done = false;

         if (rem[i] > tq) {
            cout << "Process " << p[i].id << " executes for " << tq << " units\n";
            time += tq;
            rem[i] -= tq;
         } else {
            cout << "Process " << p[i].id << " executes for " << rem[i] << " units\n";
```

```
            time += rem[i];
            rem[i] = 0;
        }
    }
  }
} while (!done);

cout << "Total time = " << time << endl;

return 0;
}
```

Output:

```
Enter number of processes: 4
Enter burst time for Process 1: 3
Enter burst time for Process 2: 5
Enter burst time for Process 3: 1
Enter burst time for Process 4: 4
Enter time quantum: 2

Execution Order:
Process 1 executes for 2 units
Process 2 executes for 2 units
Process 3 executes for 1 units
Process 4 executes for 2 units
Process 1 executes for 1 units
Process 2 executes for 2 units
Process 4 executes for 2 units
Process 2 executes for 1 units
Total time = 13

--------------------------------
Process exited after 12.8 seconds with return value 0
Press any key to continue . . .
```

9) Given two sorted linked list, merge them into single sorted list.

```
#include <iostream>
```

```
using namespace std;

struct Node {
   int data;
   Node* next;
};

void insert(Node*& head, int value) {
   Node* newNode = new Node();
   newNode->data = value;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
   } else {
      Node* temp = head;
      while (temp->next != NULL)
         temp = temp->next;
      temp->next = newNode;
   }
}

void display(Node* head) {
   while (head != NULL) {
      cout << head->data << " ";
      head = head->next;
   }
   cout << endl;
```

```
}

Node* mergeLists(Node* l1, Node* l2) {
  Node dummy;
  Node* tail = &dummy;
  dummy.next = NULL;

  while (l1 != NULL && l2 != NULL) {
    if (l1->data < l2->data) {
      tail->next = l1;
      l1 = l1->next;
    } else {
      tail->next = l2;
      l2 = l2->next;
    }
    tail = tail->next;
  }

  if (l1 != NULL)
    tail->next = l1;
  else
    tail->next = l2;

  return dummy.next;
}

int main() {
  Node* list1 = NULL;
```

```
    Node* list2 = NULL;

    insert(list1, 1);
    insert(list1, 3);
    insert(list1, 5);

    insert(list2, 2);
    insert(list2, 4);
    insert(list2, 6);

    cout << "List 1: ";
    display(list1);

    cout << "List 2: ";
    display(list2);

    Node* merged = mergeLists(list1, list2);

    cout << "Merged list: ";
    display(merged);

    return 0;
}
```

Output:

```
List 1: 1 3 5
List 2: 2 4 6
Merged list: 1 2 3 4 5 6

--------------------------------
Process exited after 1.811 seconds with return value 0
Press any key to continue . . .
```

10)Given a circular linked list, split them into two equal halves, or nearly equal if odd length.

```
#include <iostream>

using namespace std;


struct Node {

   int data;

   Node* next;

};


void insert(Node*& head, int value) {

   Node* newNode = new Node();

   newNode->data = value;


   if (head == NULL) {

      head = newNode;

      newNode->next = head;

      return;

   }


   Node* temp = head;

   while (temp->next != head) {
```

```
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->next = head;
}

void display(Node* head) {
    if (head == NULL) return;

    Node* temp = head;
    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != head);
    cout << endl;
}

void splitList(Node* head, Node*& head1, Node*& head2) {
    if (head == NULL || head->next == head) {
        head1 = head;
        head2 = NULL;
        return;
    }

    Node* slow = head;
    Node* fast = head;
```

```
    while (fast->next != head && fast->next->next != head) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (fast->next->next == head)
        fast = fast->next;

    head1 = head;
    head2 = slow->next;

    fast->next = head2;
    slow->next = head1;
}

int main() {
    Node* head = NULL;
    Node* head1 = NULL;
    Node* head2 = NULL;

    insert(head, 1);
    insert(head, 2);
    insert(head, 3);
    insert(head, 4);
    insert(head, 5);

    cout << "Original Circular List: ";
    display(head);
```

```
    splitList(head, head1, head2);

    cout << "First Half: ";
    display(head1);

    cout << "Second Half: ";
    display(head2);

    return 0;
}
```

Output:

```
Original Circular List: 1 2 3 4 5
First Half: 1 2 3
Second Half: 4 5

--------------------------------
Process exited after 2.242 seconds with return value 0
Press any key to continue . . .
```